

MAST32001 Computational Statistics - Lecture 8

Cross-validation, model selection, and the bias–variance trade-off

Klaus Nordhausen

University of Helsinki

WS 2026

Motivation

Bootstrap versus cross-validation

Both are resampling ideas, but they usually answer different questions.

Bootstrap

- ▶ How variable is an estimator?
- ▶ What is its bias or standard error?
- ▶ What confidence interval can we construct?

Cross-validation

- ▶ How well will a fitted procedure predict new observations?
- ▶ Which tuning parameter or model should we choose?
- ▶ How much optimism is hidden in the training error?

Bootstrap: uncertainty of an estimator	CV: out-of-sample predictive performance
--	--

The prediction problem

Suppose

$$Y = f(X) + \varepsilon, \quad E(\varepsilon \mid X) = 0.$$

We observe training data

$$\mathcal{D} = \{(X_i, Y_i) : i = 1, \dots, n\}$$

and construct a fitted rule

$$\hat{f}_{\mathcal{D}}.$$

What we really care about is performance on a **new** observation (X_0, Y_0) from the same population:

$$R(\hat{f}_{\mathcal{D}}) = E \left[L\{Y_0, \hat{f}_{\mathcal{D}}(X_0)\} \mid \mathcal{D} \right].$$

For squared-error loss,

$$L(y, \hat{y}) = (y - \hat{y})^2.$$

Training error is optimistic

The training error is

$$\hat{R}_{\text{train}} = \frac{1}{n} \sum_{i=1}^n L\{Y_i, \hat{f}_{\mathcal{D}}(X_i)\}.$$

But the same observations were used to **choose** $\hat{f}_{\mathcal{D}}$.

As model flexibility increases, training error can only improve or remain unchanged for many nested model classes.

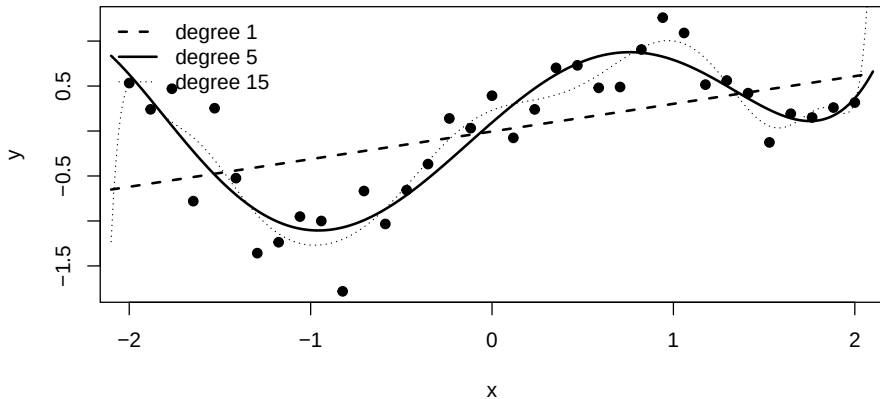
That does not imply that prediction on new data improves.

Good fit to observed data $\neq$ good prediction of new data

Underfitting and overfitting

- ▶ **Underfitting:** the model is too restricted to capture important structure.
- ▶ **Overfitting:** the model adapts too strongly to accidental features of the observed sample.

Same data, different flexibility



Bias and variance

Why can a flexible model predict badly?

Fix a point x_0 . Across repeated training samples, $\hat{f}(x_0)$ is random.

For squared-error prediction and

$$Y_0 = f(x_0) + \varepsilon_0, \quad \text{Var}(\varepsilon_0) = \sigma^2,$$

the expected prediction error decomposes as

$$E\{(Y_0 - \hat{f}(x_0))^2\} = \sigma^2 + \text{Bias}\{\hat{f}(x_0)\}^2 + \text{Var}\{\hat{f}(x_0)\}.$$

The three terms are:

- ▶ irreducible noise;
- ▶ squared bias;
- ▶ variance.

Deriving the bias–variance decomposition

Write

$$Y_0 - \hat{f} = \{Y_0 - f\} + \{f - E(\hat{f})\} + \{E(\hat{f}) - \hat{f}\}.$$

After squaring and taking expectations, the cross-products vanish:

$$E\{(Y_0 - \hat{f})^2\} = E\{(Y_0 - f)^2\} + \{f - E(\hat{f})\}^2 + E[\{\hat{f} - E(\hat{f})\}^2].$$

Therefore

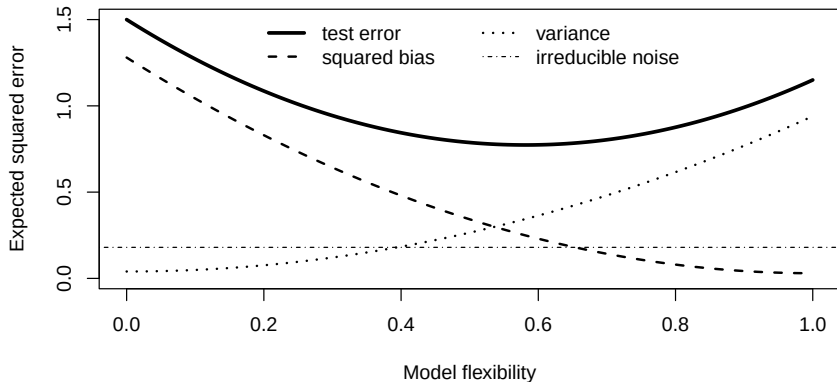
$$E\{(Y_0 - \hat{f})^2\} = \sigma^2 + \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f}).$$

This is an expectation over **repeated training samples** and a new response.

Flexibility changes bias and variance

Typically, as model flexibility increases:

- ▶ bias decreases;
- ▶ variance increases;
- ▶ training error decreases;
- ▶ test error may first decrease and then increase.



Bias and variance are not properties of a fitted curve

A common misconception:

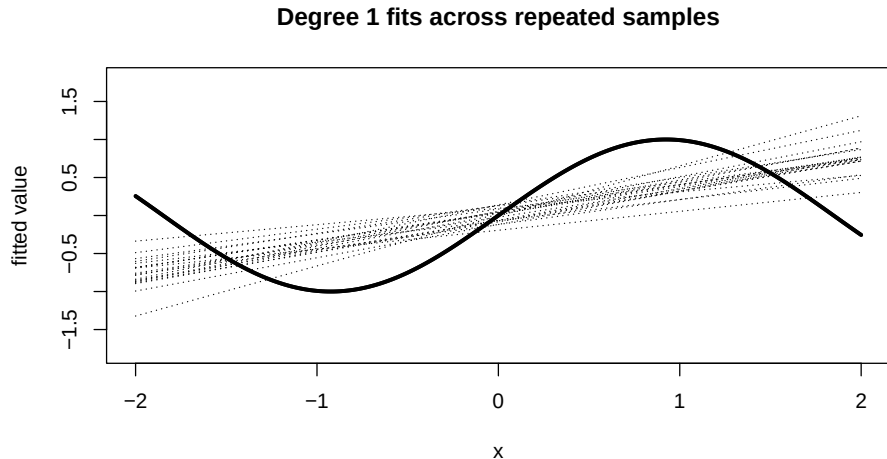
“This fitted curve looks wiggly, therefore it has high variance.”

Variance refers to what happens if we repeatedly obtain new training samples and refit the procedure.

A highly flexible method tends to have high variance because small changes in the training data can cause large changes in the fitted function.

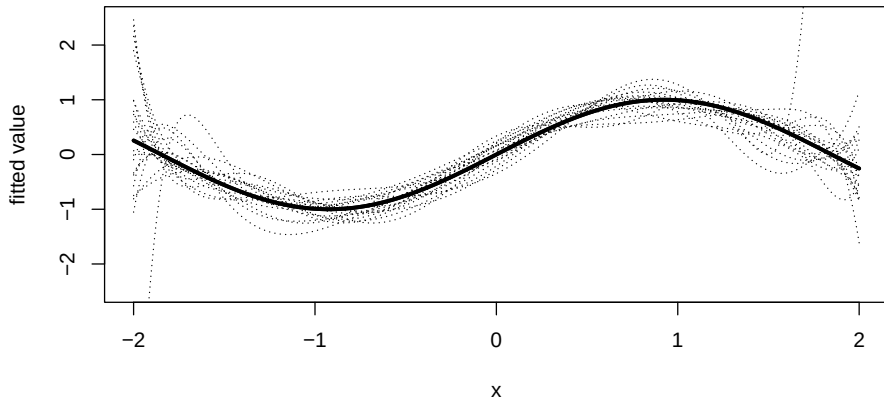
Bias also concerns the **procedure**, not merely one realized fit.

Simulation: repeated training samples



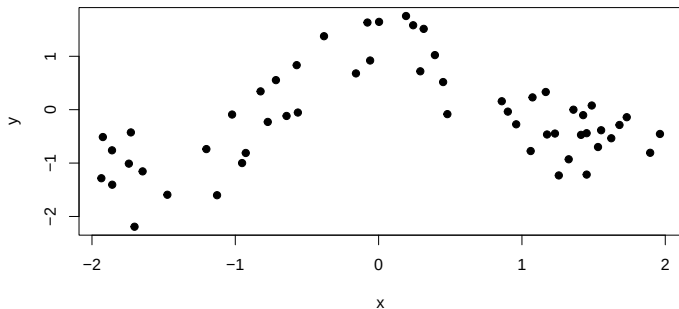
Simulation: more flexible fits

Degree 9 fits across repeated samples



Active task 1: choose the model before computing CV

We observe:



Candidate models are polynomial regressions of degrees

1, 2, 3, 5, 10.

Active task 1: choose the model before computing CV

In pairs:

1. Which degree would you choose from the plot on the previous slide?
2. Rank the five models from **highest expected bias** to lowest.
3. Rank them from **highest expected variance** to lowest.
4. Which model do you expect to have the smallest training MSE?
5. Is that also your prediction for test MSE?

Do not fit the models yet.

Active task 1: check with repeated train/test experiments

The data from the figure can be generated as

```
set.seed(55)
x_task <- sort(runif(55, -2, 2))
y_task <- cos(2 * x_task) + 0.25 * x_task + rnorm(55, sd = 0.45)
```

Active task 1: check with repeated train/test experiments

```
degrees <- c(1, 2, 3, 5, 10)
one_split <- function() {
  ind <- sample(seq_along(x_task), 35)
  train <- data.frame(x = x_task[ind], y = y_task[ind])
  test  <- data.frame(x = x_task[-ind], y = y_task[-ind])
  sapply(degrees, function(d) {
    fit <- lm(y ~ poly(x, d, raw = TRUE), data = train)
    pred <- predict(fit, newdata = test)
    mean((test$y - pred)^2)
  })
}
set.seed(56)
mse_many <- replicate(200, one_split())
rowMeans(mse_many)
apply(mse_many, 1, sd)
```

Questions:

1. Was your ranking reasonable?
2. Which models have especially unstable test error?
3. Why can a single validation split give a misleading winner?

Active task 1: check with repeated train/test experiments

```
rowMeans(mse_many)
```

```
## [1] 0.8484836 0.4614292 0.4983224 0.3117916 1.1142442
```

```
apply(mse_many, 1, sd)
```

```
## [1] 0.18500530 0.09660407 0.13630275 0.15798162 4.63750621
```

From holdout to cross-validation

Validation set approach

Split the observations into:

- ▶ a **training set**, used to fit the model;
- ▶ a **validation/test set**, used to estimate prediction error.

For a test set V ,

$$\hat{R}_V = \frac{1}{|V|} \sum_{i \in V} L\{Y_i, \hat{f}_{\mathcal{D} \setminus V}(X_i)\}.$$

This is conceptually clean: the observations used for evaluation were not used for fitting.

Why one split is unsatisfactory

Two problems from the old lecture remain central:

1. **Split variability:** the result can depend strongly on which observations happen to enter the training and validation sets.
2. **Reduced training size:** the model is trained on fewer observations than will eventually be available.

Repeated splitting can address the first issue, but not completely the second.

Cross-validation organizes repeated holdout systematically.

K -fold cross-validation

Partition the indices into K disjoint folds

$$I_1, \dots, I_K.$$

For each k :

1. fit the procedure using all observations outside I_k ;
2. predict observations in I_k ;
3. compute their losses.

The CV estimate is

$$\hat{R}_{\text{CV}} = \frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} L\{Y_i, \hat{f}^{(-k)}(X_i)\}.$$

Each observation is evaluated exactly once out of sample.

Why weight by observations?

If folds have equal size, one can equivalently average the fold-specific MSEs.

But in general,

$$\frac{1}{K} \sum_{k=1}^K \text{MSE}_k$$

and

$$\frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} (Y_i - \hat{Y}_i^{(-k)})^2$$

need not be identical when fold sizes differ.

The observation-weighted form directly estimates the average loss across held-out observations.

Leave-one-out cross-validation

LOOCV is $K = n$.

For observation i , fit without observation i :

$$\hat{f}^{(-i)}.$$

Then

$$\hat{R}_{\text{LOO}} = \frac{1}{n} \sum_{i=1}^n L\{Y_i, \hat{f}^{(-i)}(X_i)\}.$$

Advantages:

- ▶ almost the full sample is used for each fit;
- ▶ no random fold assignment.

Disadvantages:

- ▶ potentially expensive;
- ▶ the fitted training sets are extremely similar, so the errors can be strongly correlated.

LOOCV for ordinary least squares

For OLS there is a useful shortcut.

Let

$$e_i = Y_i - \hat{Y}_i$$

be the ordinary residual and h_{ii} the i -th leverage.

The leave-one-out residual is

$$e_i^{(-i)} = \frac{e_i}{1 - h_{ii}}.$$

Therefore

$$\hat{R}_{\text{LOO}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{e_i}{1 - h_{ii}} \right)^2.$$

No n separate regressions are required.

Why does leverage appear?

A high-leverage observation has a larger effect on its own fitted value.

Its ordinary residual

$$e_i = Y_i - \hat{Y}_i$$

can therefore look artificially small.

Removing the observation changes the fit more strongly when h_{ii} is large.

The correction

$$\frac{1}{1 - h_{ii}}$$

turns the ordinary residual into the leave-one-out prediction residual.

Choosing K

Common choices are $K = 5$ or $K = 10$.

There is no universal theorem saying that 5 or 10 is always optimal.

Changing K changes:

- ▶ training-set size;
- ▶ computational cost;
- ▶ dependence among fitted models;
- ▶ bias and variance of the CV error estimate.

LOOCV has very large training sets but highly overlapping fits.

Smaller K uses smaller training sets but can give less strongly correlated validation errors.

Repeated K -fold CV

Ordinary K -fold CV still depends on a random partition.

A simple remedy is to repeat the entire K -fold procedure several times with new fold assignments.

This estimates how sensitive the result is to the partition.

But remember:

repeating CV reduces split noise; it does not create new data

CV estimates the performance of a training procedure

Suppose 10-fold CV is used.

Each fitted model uses approximately $0.9n$ observations.

Thus CV directly evaluates a procedure trained on about $0.9n$ observations, not exactly the final model fitted on all n .

Usually we select the model/tuning parameter by CV and then **refit the chosen procedure on all available training data**.

What exactly are we selecting?

Model assessment versus model selection

Model assessment

Estimate the predictive performance of a specified procedure.

Model selection

Use the data to choose among procedures:

$$\hat{m} = \arg \min_m \hat{R}_{CV}(m).$$

These are different tasks.

Once CV has been used to choose $\hat{m}$, the minimum CV error is itself optimistically selected.

Selection creates another layer of overfitting

Imagine comparing 1000 tuning parameters.

Even if their true risks are very similar, some will have unusually low CV errors simply by chance.

Choosing the minimum exploits this noise.

Therefore

$$\min_m \hat{R}_{CV}(m)$$

is generally too optimistic as an estimate of the performance of the **entire selection procedure**.

This motivates nested cross-validation.

Nested cross-validation

Use two levels:

- ▶ **inner CV:** choose hyperparameters/model;
- ▶ **outer CV:** estimate performance of the complete selection procedure.

For each outer fold:

1. hold out the outer test fold;
2. choose the model using only the remaining data and inner CV;
3. fit the selected model on the outer training data;
4. evaluate once on the untouched outer fold.

The outer test fold must not influence model selection.

Data leakage

Leakage occurs when information from the validation/test observations influences training.

Examples:

- ▶ standardizing variables using the full dataset before CV;
- ▶ selecting variables using all outcomes before CV;
- ▶ imputing missing values using the full dataset;
- ▶ tuning a model and reporting performance on the same test set.

All preprocessing that learns from data should generally be performed **inside each training fold**.

Prediction loss

Loss functions depend on the goal

Regression:

$$\text{MSE} = \frac{1}{n} \sum_i (Y_i - \hat{Y}_i)^2, \quad \text{MAE} = \frac{1}{n} \sum_i |Y_i - \hat{Y}_i|.$$

Classification:

- ▶ misclassification rate;
- ▶ log loss;
- ▶ Brier score;
- ▶ sensitivity/specificity when scientifically relevant.

Different losses answer different questions.

A model can be best under one loss and not under another.

Classification: accuracy can be misleading

Suppose 95% of observations belong to class 0.

A classifier that always predicts 0 has

95% accuracy,

but zero sensitivity for class 1.

Therefore the performance measure should reflect the scientific or practical cost of errors.

Cross-validation cannot rescue an inappropriate loss function.

AIC, BIC, and model selection

CV is not the only model-selection idea

For likelihood-based parametric models, information criteria provide another route.

Let

$$\ell(\hat{\theta})$$

be the maximized log-likelihood and p the number of estimated parameters.

AIC:

$$\text{AIC} = -2\ell(\hat{\theta}) + 2p.$$

BIC:

$$\text{BIC} = -2\ell(\hat{\theta}) + p \log n.$$

Smaller values are preferred.

Both trade goodness of fit against model complexity.

Why penalize complexity?

The maximized likelihood always tends to improve as parameters are added in nested models.

Without a penalty, the largest model wins.

AIC and BIC use

$$-2\ell(\hat{\theta}) + \text{penalty}.$$

The first term rewards fit; the second penalizes flexibility.

For $n > e^2 \approx 7.4$,

$$p \log n > 2p,$$

so BIC penalizes additional parameters more strongly than AIC.

AIC and BIC have different goals

AIC is motivated by estimating out-of-sample predictive discrepancy, closely related to Kullback–Leibler risk.

BIC arises from a large-sample approximation to Bayesian model evidence under particular prior/model assumptions.

Consequently:

- ▶ AIC is often associated with predictive efficiency;
- ▶ BIC more strongly favors parsimonious models and, under suitable conditions, can consistently identify a true finite-dimensional model.

They should not be treated as interchangeable formulas with different constants.

AIC versus cross-validation

For regular likelihood models, AIC and leave-one-out CV are closely related asymptotically.

But CV is more general:

- ▶ arbitrary prediction algorithms;
- ▶ arbitrary losses;
- ▶ tuning parameters without a simple likelihood.

AIC requires a likelihood and a meaningful parameter count.

BIC additionally targets a different model-selection principle.

Adjusted R^2 and training R^2

In linear regression,

$$R^2$$

never decreases when predictors are added.

So maximizing training R^2 is not a useful model-selection rule.

Adjusted R^2 introduces a complexity correction:

$$R_{\text{adj}}^2 = 1 - \frac{\text{RSS}/(n - p - 1)}{\text{TSS}/(n - 1)}.$$

It is useful descriptively, but it is not a direct estimate of out-of-sample prediction error.

One-standard-error rule

Sometimes several models have almost indistinguishable CV performance.

Instead of choosing the absolute minimum, one can choose the **simplest** model whose estimated CV error is within one standard error of the minimum.

This is the one-standard-error rule.

Its purpose is not a theorem that the simpler model is correct.

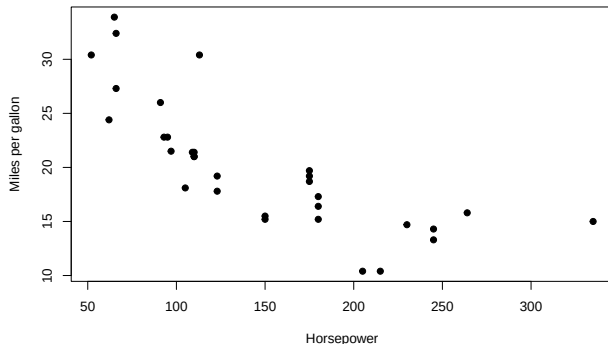
It is a pragmatic way to avoid selecting extra complexity for a tiny, noisy estimated improvement.

Active model-selection example

Real data: mtcars

We predict fuel consumption `mpg` from horsepower `hp`.

Candidate polynomial degrees: $1, \dots, 8$.



Before computing anything:

1. What degree seems plausible?
2. Would you trust degree 8 with only 32 observations?
3. Which criterion do you expect to prefer the simplest model: AIC or BIC?

Computing 10-fold CV

```
kfold_mse <- function(data, degree, K = 10, fold_id) {  
  losses <- numeric(nrow(data))  
  
  for (k in seq_len(K)) {  
    train <- data[fold_id != k, ]  
    test  <- data[fold_id == k, ]  
    fit <- lm(  
      mpg ~ poly(hp, degree, raw = TRUE),  
      data = train  
    )  
    pred <- predict(fit, newdata = test)  
    losses[fold_id == k] <- (test$mpg - pred)^2  
  }  
  mean(losses)  
}
```

Comparing CV, AIC, and BIC

```
degrees <- 1:8
set.seed(77)
fold_id <- sample(rep(1:10, length.out = nrow(mtcars)))
cv <- sapply(
  degrees, function(d) kfold_mse(mtcars, d, K = 10, fold_id = fold_id)
)
fits <- lapply(
  degrees, function(d) lm(mpg ~ poly(hp, d, raw = TRUE), data = mtcars)
)
aic <- sapply(fits, AIC)
bic <- sapply(fits, BIC)
```

Comparing CV, AIC, and BIC

```
data.frame(degree = degrees, CV = cv, AIC = aic, BIC = bic)
```

##	degree	CV	AIC	BIC
## 1	1	17.24678	181.2386	185.6358
## 2	2	10.19321	167.6023	173.4652
## 3	3	11.39883	169.0112	176.3399
## 4	4	65.10928	171.0009	179.7953
## 5	5	449.22678	172.7871	183.0473
## 6	6	1032.45858	172.0192	183.7451
## 7	7	21579.91438	173.9883	187.1800
## 8	8	260760.54982	175.0306	189.6880

Do not compare the numerical scales

CV MSE, AIC, and BIC are on different scales.

Do **not** compare, for example,

$$CV = 20 \quad \text{with} \quad AIC = 170.$$

Instead compare models **within each criterion**:

- ▶ which degree minimizes CV?
- ▶ which degree minimizes AIC?
- ▶ which degree minimizes BIC?

Different criteria can select different models because their targets differ.

Active task 2: model selection is uncertain

Using the `mtcars` example:

1. Run the 10-fold CV once and record the selected degree.
2. Repeat with five different random fold assignments.
3. Does the selected degree change?
4. Compare the selected degrees with AIC and BIC.
5. If degrees 2 and 3 have nearly identical CV errors, which would you report and why?
6. What would be wrong with choosing the degree by CV and then reporting the same minimum CV error as an unbiased estimate of final predictive performance?

Active task 2: model selection is uncertain

```
select_degree <- function(seed) {  
  set.seed(seed)  
  fold_id <- sample(rep(1:10, length.out = nrow(mtcars)))  
  
  cv <- sapply(  
    1:8, function(d) kfold_mse(  
      mtcars, d, K = 10, fold_id = fold_id  
    )  
  )  
  c(degree = which.min(cv), min_CV = min(cv))  
}
```

Active task 2: model selection is uncertain

```
sapply(1:5, select_degree)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]  
## degree  2.00000  3.00000  2.00000  3.00000  3.00000  
## min_CV  10.53939 10.10361 10.67163 10.1649 10.45455
```

Active task 2: discussion

The point is not that CV is unreliable.

The point is that

$$\hat{R}_{CV}$$

is itself an estimate and therefore has sampling/splitting variability.

If several models are close, the exact winner may be unstable.

Useful responses include:

- ▶ repeated CV;
- ▶ a one-standard-error rule;
- ▶ scientific preference for a simpler model;
- ▶ nested CV when an unbiased assessment of the full selection procedure is required.

Dependence and cross-validation

Ordinary random folds assume exchangeable observations

Random K -fold CV is natural for iid observations.

But if observations are dependent, random splitting can leak information.

Examples:

- ▶ time series;
- ▶ repeated measurements from the same person;
- ▶ clustered observations;
- ▶ spatially nearby observations.

The folds should respect the structure of the prediction problem.

Time series: do not predict the past from the future

For forecasting, randomly mixing observations across folds is usually inappropriate.

A more realistic scheme is **forward validation**:

$$\underbrace{1, \dots, t}_{\text{training}} \longrightarrow \underbrace{t+1, \dots, t+h}_{\text{validation}}.$$

Then move the origin forward and repeat.

The validation design should mimic how the fitted model will actually be used.

Grouped data

If several observations come from the same person, school, hospital, or experimental unit, random observation-level folds may put related observations in both training and validation sets.

If the goal is prediction for **new groups**, hold out whole groups.

The resampling unit should match the intended generalization unit.

Practical workflow

A defensible CV workflow

1. Define the prediction target and loss.
2. Decide what constitutes an independent/generalization unit.
3. Keep a final test set aside if an external final assessment is needed.
4. Put preprocessing and model fitting inside the CV loop.
5. Compare candidate models/tuning parameters using the same folds.
6. Examine uncertainty/stability, not only the minimum.
7. Select a procedure.
8. Refit the selected procedure using all available training data.
9. Evaluate once on untouched test data, if available.

Common mistakes

- ▶ reporting training error as prediction error;
- ▶ preprocessing before splitting;
- ▶ repeatedly looking at the test set;
- ▶ using ordinary random CV for dependent data;
- ▶ comparing models on different fold assignments unnecessarily;
- ▶ choosing a huge model because its CV error is microscopically smaller;
- ▶ treating the minimum CV error after extensive tuning as unbiased;
- ▶ assuming AIC, BIC, and CV answer exactly the same question.

What should you remember?

Cross-validation estimates out-of-sample performance by repeatedly separating **fitting** from **evaluation**.

Bias and variance explain why neither the simplest nor the most flexible procedure must predict best.

Prediction requires evaluating a fitted procedure on data it did not fit.

For model selection:

- ▶ CV directly targets a chosen predictive loss;
- ▶ AIC balances likelihood fit with a $2p$ penalty;
- ▶ BIC uses a stronger $p \log n$ penalty and a different inferential target;
- ▶ selection itself can overfit, motivating nested CV.

And always ask:

Does my validation scheme mimic the way the model will be used?